

Personal Finance Manager

Project Report

CS160 - Introduction to Computer Science

Team Members:

Phạm Quốc Bảo (ID: 25125080)

Lê Nguyễn Hải Đăng (ID: 25125081)

Nguyễn Quốc Bảo (ID: 25125079)

Date of Submission:

December 28, 2025

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Objectives and Goals	3
2	Requirements Specification	3
2.1	Functional Requirements	3
2.1.1	Dashboard & Visualization	3
2.1.2	Transaction Management	3
2.1.3	Recurring Transaction Automation	4
2.1.4	Master Data Management (CRUD)	4
2.1.5	Reporting & Statistics	4
2.2	Non-Functional Requirements	4
3	Design Document	4
3.1	High-Level Architecture	4
3.2	Class Description	5
3.3	Key Algorithms and Logic	5
3.3.1	Recurring Transaction Logic	5
3.3.2	Custom Data Structures	5
4	Implementation Details	6
4.1	Code Organization	6
4.2	Libraries and Technology Stack	6
5	Detailed Frontend Implementation	7
5.1	Overview	7
5.2	Component map and responsibilities	7
5.3	Data and event flow	8
5.4	Validation and error reporting	9
5.5	UI patterns and conventions	9
5.6	Extensibility and testing	9
6	Detailed Backend Implementation	10
6.1	Transaction Management	10
6.1.1	Data Model (Structs)	10
6.1.2	Adding Transactions (IncomeArray::addTransaction)	10
6.2	Recurring Transactions Management	11
6.2.1	Data Model (RecurringTask)	11
6.2.2	Execution Logic (shouldExecute)	11
6.3	Master Data Management	12
6.3.1	Data Structure: Hash Table Node	13
6.3.2	Adding Categories (Create)	13
6.3.3	Editing Categories (Update)	14
6.3.4	Deleting Categories (Delete)	14
6.4	Statistics & Reporting	15
6.4.1	Time-based Statistics	15
6.4.2	Time & Wallet-based Statistics	16

6.4.3	Annual Breakdown (Income/Expense)	17
6.4.4	Detailed Transaction Log (History)	19
6.4.5	Architecture: Data Synchronization Frontend Integration	19
6.5	Data Persistence	20
6.5.1	Binary Serialization (File Storage)	20
6.5.2	Session Persistence (Loading Logic)	20
7	Testing	21
7.1	Testing Strategy	21
7.2	Test Results	21
7.3	Performance Optimization	22
7.4	Known Issues and Limitations	23
8	Future Improvements	23
8.1	Technical Architecture & Refactoring	23
8.2	User Interface (UI) and Experience	23
8.3	Functional Enhancements	24
8.4	Security and Persistence	24
9	Demo Videos	25
10	Work Division	25
11	Collaboration Tools Used	25

1 Project Overview

1.1 Introduction

The **Personal Finance Manager** is a robust, console-based application designed to help users track their daily financial activities, manage multiple wallets, and analyze their financial health through detailed statistics.

1.2 Objectives and Goals

The primary objective of this project is to develop a modular financial system that bridges the gap between complex accounting software and simple spreadsheet tracking. The key goals include:

- **Financial Awareness:** To provide users with an immediate, real-time view of their total assets and individual wallet balances via an interactive dashboard.
- **Automation:** To reduce the manual burden of tracking regular payments (e.g., rent, subscriptions) through an automated recurring transaction engine.
- **Data Integrity:** To ensure the logical consistency of financial data by preventing the deletion of active entities (Wallets, Categories) referenced by existing transactions.
- **Performance:** To utilize efficient custom data structures (Hash Tables and Dynamic Arrays) rather than relying solely on high-level libraries, demonstrating an understanding of low-level memory management and algorithmic optimization.

2 Requirements Specification

This section details the functional and non-functional requirements implemented in the application.

2.1 Functional Requirements

2.1.1 Dashboard & Visualization

- The system shall display the **Total Balance** aggregated across all wallets immediately upon launch.
- The system shall list individual wallets with their specific balances, color-coded to indicate financial health (e.g., standard vs. zero/negative balance).

2.1.2 Transaction Management

- Users must be able to record **Income** and **Expense** transactions.
- Inputs must include: Amount, Wallet, Category/Source, Date, and Description.
- **Validation:** The system must prevent entering expenses that exceed the selected wallet's current balance (Error Code 9).

- **Input Constraints:** The system must validate date formats (DD/MM/YYYY) and ensure monetary values are positive integers.

2.1.3 Recurring Transaction Automation

- Users shall be able to define recurring incomes or expenses with a Start Date and an optional End Date.
- Upon application startup, the system must automatically check for due recurring tasks and generate the corresponding transactions.
- The system must ensure no duplicate transactions are generated for the same cycle (e.g., the same month).

2.1.4 Master Data Management (CRUD)

- Users shall be able to Create, Read, Update, and Delete (CRUD) **Wallets**, **Income Sources**, and **Expense Categories**.
- **Data Integrity Protection:** The system must block the deletion of any Master Data entity if it is currently referenced by an existing transaction or recurring task, preventing "orphaned" data records.

2.1.5 Reporting & Statistics

- **Time-Based Analysis:** Generate reports for specific date ranges (From/To).
- **Wallet-Based Analysis:** Show total income and expense per wallet.
- **Annual Overview:** Provide a summary of financial activity across selected years.
- **Breakdowns:** Calculate percentage distributions for Income Sources and Expense Categories.

2.2 Non-Functional Requirements

- **Persistence:** All data must be saved to local binary files (`.dat`) upon exit and reloaded upon startup.
- **Responsiveness:** The GUI must handle user interactions (clicks, typing) without perceptible lag (maintaining 60 FPS).
- **Reliability:** The system must include a fail-safe Console Interface (CLI) in case the GUI fails to initialize.

3 Design Document

3.1 High-Level Architecture

The application follows a **Layered Architecture** pattern, strictly separating the Presentation Layer (GUI) from the Data Layer (Backend Logic).

1. **Presentation Layer (GUI):** Built using Raylib. This layer handles window rendering, input polling, and event dispatching. It contains classes like `App`, `TransactionWindow`, and reusable custom widgets (`Button`, `Dropdown`).
2. **Data Layer (Backend):** Manages the business logic and data storage. It is encapsulated by the `DataManager` class, which acts as a centralized repository for all data structures.
3. **Persistence Layer:** Handles the serialization of C++ structs into binary files for permanent storage.

3.2 Class Description

The system relies on the following key class relationships:

- **App:** The entry point and state machine. It owns an instance of `DataManager` and manages the switching between different `Screen` states (`Dashboard`, `Transaction`, `MasterData`).
- **DataManager:** The coordinator class that aggregates all specific data managers. It owns:
 - `WalletTable`, `CategoryTable`, `IncomeSourceTable` (Inherit from `NameTable`).
 - `IncomeArray`, `ExpenseArray` (Dynamic Arrays for transactions).
 - `RecurringManager` (Logic for automation).
- **NameTable:** A custom implementation of a Hash Table with Chaining, used for efficient ID-to-Name lookups ($O(1)$ average complexity).

3.3 Key Algorithms and Logic

3.3.1 Recurring Transaction Logic

The recurring engine uses a state-tracking mechanism based on the `lastExecuted` date.

1. On startup, the system iterates through all active recurring tasks.
2. It compares the `Current Date` vs. `Start Date` and `Last Executed Date`.
3. **Condition:** If `CurrentMonth > LastExecutedMonth`, and `CurrentDay  $\geq$  StartDay`, a new transaction is generated.
4. The system validates wallet balance before auto-deducting expenses.

3.3.2 Custom Data Structures

To demonstrate algorithmic proficiency, the project avoids standard STL containers for core storage:

- **Dynamic Arrays:** Used for transaction lists (`IncomeArray`). Implements manual memory reallocation (resizing capacity by $2x$) when the array is full.
- **Hash Tables:** Used for Master Data (`NameTable`). Implements a hashing function $key \pmod{size}$ to distribute IDs into buckets, handling collisions via Linked Lists.

4 Implementation Details

4.1 Code Organization

The project follows a modular, layered architecture designed to separate business logic/-data management from the graphical user interface. The codebase is organized into two primary directories: `src/` (Backend) and `GUI/` (Frontend).

- **Backend Logic (Data Layer):** The core functionality is encapsulated in the `DataManager` class, which serves as a central repository for all financial data. It manages custom data structures including:
 - **Custom Containers:** `NameTable` (hash table with chaining) manages relational entities like Wallets and Categories, while `IncomeArray/ExpenseArray` manage dynamic lists of transactions.
 - **Logic Modules:** `recurring.cpp` handles the automated date-checking algorithm for periodic transactions, and `statistic.cpp` provides aggregation logic for reporting.
 - **Persistence:** Custom binary serialization logic is implemented directly within the data classes to handle saving and loading state from `.dat` files.
- **Graphical User Interface (Presentation Layer):** The GUI is built upon a custom lightweight widget framework located in `GUI/src/`.
 - **App Controller:** The `App` class functions as the central state machine, managing the main game loop, initialization, and screen transitions (Dashboard ↔ Sub-windows).
 - **Window Modules:** Each functional screen is encapsulated in its own class (e.g., `TransactionWindow`, `MasterDataWindow`), which holds a reference to the `DataManager` to perform CRUD operations.
 - **Custom Widgets:** To avoid dependency on heavy UI frameworks, a custom set of UI components was developed from scratch using Raylib primitives, including `Button`, `TextInput`, `Dropdown`, and a `ScrollArea` for viewing long lists.

4.2 Libraries and Technology Stack

The application is developed using C++ (C++17 Standard), prioritizing performance and manual memory management control. The technology stack was chosen to maintain a lightweight footprint without the overhead of complex GUI frameworks like Qt or wxWidgets.

- **Raylib (v5.5):** The primary external library used for the graphical interface. Raylib handles window creation, input polling (mouse/keyboard), and low-level 2D rendering (shapes, text, textures). Its immediate-mode rendering style allowed for the rapid development of the custom UI component system.
- **C++ Standard Library (STL):** Utilized selectively for string manipulation (`std::string`),

file stream handling (`std::fstream`) for binary I/O, and `std::function` for handling callback logic in UI buttons.

- **Custom Data Structures:** Instead of relying heavily on STL containers like `std::vector` or `std::map`, the project implements custom dynamic arrays and hash tables. This approach provides fine-grained control over memory allocation and resizing logic, tailored specifically for the transactional data models used in the application.

5 Detailed Frontend Implementation

5.1 Overview

The frontend (GUI) is implemented as a small, self-contained widget framework built on top of Raylib. It follows an immediate-mode render model: each frame the application polls input and calls `Update()` and `Draw()` for the current screen and its widgets. The GUI is organized into three layers:

- **Top-level controller:** `App` orchestrates application lifecycle, screen transitions and persistence.
- **Screens / Windows:** Each major screen (`Dashboard`, `TransactionWindow`, `RecurringWindow`, `MasterDataWindow`, `StatisticWindow`) implements its own view, UI layout and form handling.
- **Widget library:** Reusable UI primitives (`Button`, `Dropdown`, `TextInput`, `ScrollArea`, `Dialog`) provide the look-and-feel and basic input behavior used by all screens.

The GUI delegates business logic and persistence to the `DataManager`, which acts as the single source of truth for all model data used by the frontend.

5.2 Component map and responsibilities

Below is a short mapping of the important GUI source files to their responsibilities and typical interactions.

Application / lifecycle

- `mainGUI.cpp` — entrypoint. Calls Raylib initialization, creates the `App` instance, runs the main loop and performs shutdown.
- `App.cpp` / `App.h` — central orchestrator. Loads data via `DataManager::LoadAllData()`, forwards per-frame calls (`Update/Draw`) to the active window, and calls `SaveAllData()` on shutdown. `App` holds instances (or lazily creates) the sub-windows.

Windows (screens)

Each window implements at minimum: initialization, `Update()`, `Draw()`, and data binding.

- `TransactionWindow.cpp` / `TransactionWindow.h`
Purpose: two-column overview of Incomes and Expenses, plus Add Income / Add

Expense dialogs. Handles validation for forms and displays the result via shared error indicator helpers.

Dependencies: `DataManager`, `Dropdown`, `TextInput`, `Dialog`, `ScrollArea`, `Helper`, `LayoutHelper`.

- `MasterDataWindow.cpp / MasterDataWindow.h`
Purpose: CRUD UI for wallets, income sources and categories.
Dependencies: `NameTable` API via `DataManager`, `Dialog`, `ScrollArea`.
- `RecurringWindow.cpp / RecurringWindow.h`
Purpose: create/edit/list recurring tasks; uses the same form widgets as `TransactionWindow` and delegates actual recurring processing to the `DataManager/RecurringManager`.
Dependencies: `DataManager`, `Dialog`, `Dropdown`, `TextInput`.
- `StatisticWindow.cpp / StatisticWindow.h`
Purpose: present aggregated data (reports, breakdowns) computed by the statistic module.
Dependencies: `DataManager` APIs and helper formatting utilities.
- `dashboard.cpp / dashboard.h`
Purpose: landing overview — aggregate balances, recent activity and navigation buttons.
Dependencies: `DataManager` for aggregated values and widget primitives for navigation.

Widget library and helpers

- `button.cpp / button.h` — clickable control with hover/click logic and a `std::function` callback. Used throughout the UI.
- `Dropdown.cpp / Dropdown.h` — compact selection control with overlayed list rendering.
- `TextInput.cpp / TextInput.h` — single-line input with caret and basic editing.
- `ScrollArea.cpp / ScrollArea.h` — provides clipping, offset and scrollbar for long lists.
- `Dialog.cpp / Dialog.h` — modal container for inputs and confirm dialogs.
- `Helper.cpp / Helper.h` — formatting utilities (currency, date), and `DrawFormErrorTextIndicator()` used by dialogs to display validation messages.
- `LayoutHelper.cpp / LayoutHelper.h` — layout helpers: center rect, grid creation and reusable spacing constants.

5.3 Data and event flow

A typical interaction follows this pattern (example: add expense):

1. User clicks the `Add Expense Button` on the `Dashboard` or `TransactionWindow`.
2. App or `TransactionWindow` opens the `Dialog` containing `TextInput` and selection `Dropdown` widgets for wallet/category.

3. User fills values and clicks the **Add Button**. The **Dialog** collects input strings and calls the window's submit handler (e.g., `TransactionWindow::SubmitExpense()`).
4. The window performs client-side validation (parse amount, ensure wallet/category selected) and then queries the model for business checks (e.g., wallet balance).
5. On success, the window calls **DataManager** / **Income/Expense** arrays to add the transaction and then requests `DataManager::SaveAllData()` (or lets App save on shutdown). The UI shows a confirmation dialog.
6. On failure, the window sets an **errorID** and the dialog draws an error message via `DrawFormErrorTextIndicator()`.

5.4 Validation and error reporting

The GUI uses an integer **errorID** to communicate validation failures to the drawing helper. For clarity and maintainability:

- Replace magic integers with a named **enum class** `FormError{...}` so windows and helpers share the same set of error values and messages.
- Move business validation (not enough money, wallet existence) into **DataManager** (e.g., `DataManager::HasSufficientBalance()`) so windows only display errors but do not re-implement logic.

5.5 UI patterns and conventions

- **Single source of truth:** **DataManager** holds the authoritative data; windows must call its API rather than performing file I/O directly.
- **Widget contract:** Widgets expose a minimal API: `Update()`, `Draw()`, and setters for callbacks (e.g., `SetOnClick(std::function<void()>)`)
- **Layout reuse:** Use `LayoutHelper::CreateGrid()` or central spacing constants so all dialogs/windows share consistent spacing and typography.
- **No using-directives in headers:** Headers must avoid `'using namespace std;'`. Keep scope local to implementation files.
- **File naming:** Keep header/filename casing consistent (prefer all-lowercase headers) to avoid cross-platform build issues.

5.6 Extensibility and testing

- **Make the GUI testable:** Expose test hooks so the CLI or unit tests can call core logic in **DataManager** and **statistic** without opening a window.
- **Separate targets:** Build the core model as a separate library target so both GUI and CLI link to it — this enables headless CI builds.
- **Unit tests:** Add tests for formatting helpers, balance calculation (`getWalletBalance`), and `'DataManager'` validation functions. These can run in CI on Linux/macOS without Raylib.

6 Detailed Backend Implementation

This section provides a comprehensive breakdown of the algorithms and data structures implemented to satisfy all project requirements. For each core function, we provide the actual source code, an explanation of its mechanics, the design rationale, and an efficiency analysis.

6.1 Transaction Management

The system manages Income and Expense records using a strict data model that separates storage (IDs) from presentation (Names).

6.1.1 Data Model (Structs)

Transactions are stored using compact structs ('IncomeTransaction' and 'ExpenseTransaction').

```
1 struct IncomeTransaction {
2     date day;
3     double amount;
4     std::string description;
5     int sourceID; // Stores ID (e.g., 101), not "Salary"
6     int walletID; // Stores ID (e.g., 1), not "Cash"
7 };
```

Design Rationale (ID vs. Name): We store integer IDs ('sourceID', 'walletID') instead of string names. This separation is crucial for data integrity and efficiency.

- **Consistency Normalization:** Storing IDs acts as a form of database normalization. If a user renames a wallet from "Bank" to "Chase Bank" in the Master Data, all historical transactions automatically reflect the new name because they reference the immutable ID, not the mutable string. This avoids the need to update thousands of transaction records upon a simple name change.
- **Memory Efficiency:** An 'int' typically occupies 4 bytes of memory. In contrast, a 'std::string' object has significant overhead (often 24-32 bytes plus heap allocation for the character data). For a system with thousands of transactions, storing IDs saves a substantial amount of RAM and disk space.

6.1.2 Adding Transactions (IncomeArray::addTransaction)

```
1 void IncomeArray::addTransaction(date day, double amount, int sourceID,
2     int walletID, string desc) {
3     if (count == capacity) {
4         resize(); // Geometric expansion logic
5     }
6     list[count].day = day;
7     list[count].amount = amount;
8     list[count].sourceID = sourceID;
9     list[count].walletID = walletID;
10    list[count].description = desc;
11    count++;
12 }
```

How it works: The function first checks if the current dynamic array is full. If so, it triggers a 'resize()' which doubles the array's capacity. Then, it populates the transaction struct at the 'count' index with the provided data and increments the counter.

Why this approach: We use a ****Dynamic Array**** (similar to 'std::vector') because transaction data is append-only and accessed sequentially for reporting. Arrays offer excellent cache locality compared to linked lists, making report generation significantly faster.

Efficiency Analysis:

- **Time Complexity:** $O(1)$ amortized. While 'resize()' is an $O(N)$ operation (copying all elements), it occurs infrequently due to the geometric growth strategy (doubling capacity). Therefore, the average cost per insertion remains constant.
- **Constraint Check:** Before calling this function, the system validates inputs (e.g., Amount > 0) in the UI layer to ensure data validity.

6.2 Recurring Transactions Management

This module automates monthly entries. It is designed to run automatically upon application startup to ensure "Session Persistence."

6.2.1 Data Model (RecurringTask)

```
1 struct RecurringTask {
2     // ... basic info ...
3     date startDate;      // Required
4     date endDate;        // Optional (0/0/0 if infinite)
5     date lastExecuted;   // Tracks the last successful auto-add
6 };
```

6.2.2 Execution Logic (shouldExecute)

This function enforces the logic: "Run once per month, after Start Date, before End Date."

```
1 bool RecurringManager::shouldExecute(const RecurringTask& task, date
   current) {
2     // 1. Start Date Check
3     if (compareDate(current, task.startDate) < 0) return false;
4
5     // 2. End Date Check (Optional: year != 0 means it's set)
6     if (task.endDate.year != 0 && compareDate(current, task.endDate) >
   0) return false;
7
8     // 3. Duplicate Prevention (Mechanism)
9     // Check if task already ran in the current month/year cycle
10    if (task.lastExecuted.year != 0) {
11        if (task.lastExecuted.month == current.month &&
12            task.lastExecuted.year == current.year) {
13            return false; // Already done this month
14        }
15    }
16 }
```

```

17 // 4. Target Day Logic (e.g., Handle 31st vs Feb 28th)
18 int daysInCurrentMonth = getDaysInMonth(current.month, current.year)
19 ;
20 int targetDay = task.startDate.day;
21 if (targetDay > daysInCurrentMonth) targetDay = daysInCurrentMonth;
22 return current.day >= targetDay;
23 }

```

How it works: This function acts as a gatekeeper. It takes a recurring task and the current date as input.

1. It first verifies if the current date is within the valid lifespan of the recurring task (after start, before end).
2. It checks for an optional End Date. The system verifies if the task has an expiration date (indicated by `endDate.year != 0`). If an end date is set and the current date has passed it, the function returns false, effectively stopping the recurring task forever.
3. Crucially, it checks the 'lastExecuted' field. If the stored month and year match the current date, it means the task has already run for this cycle, so it returns 'false'.
4. It handles edge cases like scheduling a payment for the 31st. If the current month (e.g., February) has fewer days, it caps the target day to the last day of the month.

Why this approach: This "state-based" logic is robust for a desktop application that isn't always running. By storing 'lastExecuted', the system can correctly determine if it missed a scheduled transaction simply by checking the date upon startup, without needing a background service.

Efficiency Analysis:

- **Time Complexity:** $O(K)$ where K is the number of recurring tasks. Since K is usually small (<50), iterating through this list on startup is instantaneous.

6.3 Master Data Management

Managed by the 'NameTable' class, this module handles shared data categories. We implemented a ****Hash Table with Separate Chaining**** to manage the ID-to-Name mapping efficiently.

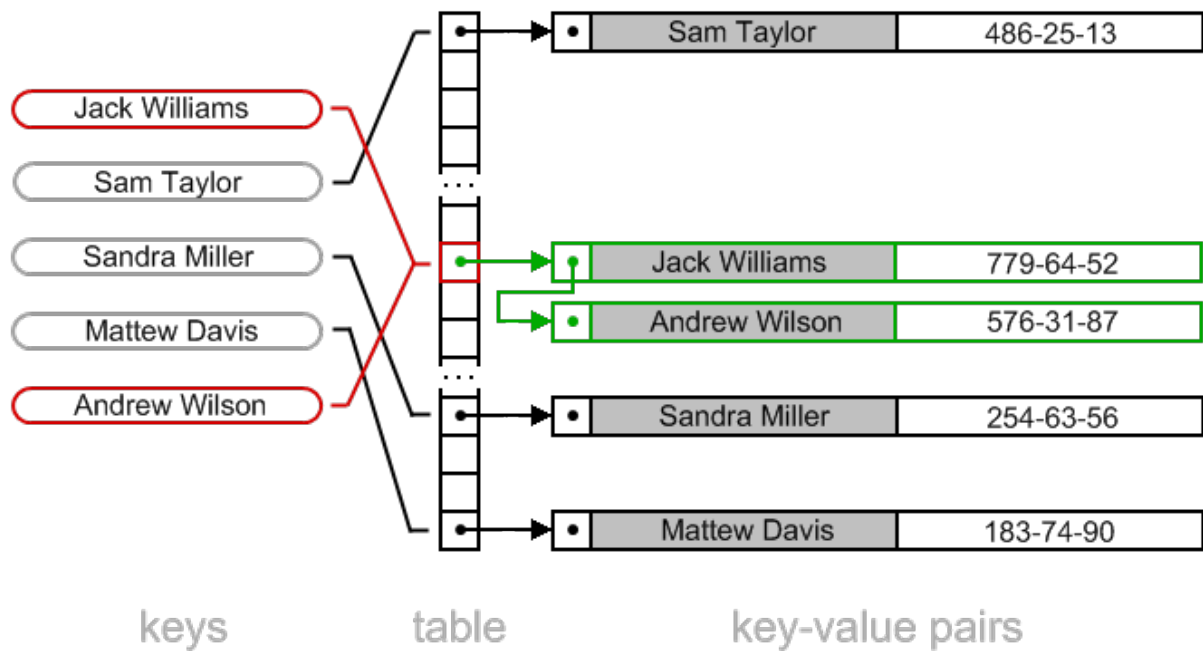


Figure 1: Example of a hash table

6.3.1 Data Structure: Hash Table Node

```

1 struct Node {
2     int id;
3     std::string name;
4     Node* next; // Pointer for linked list chaining
5     Node(int id, std::string name) : id(id), name(name), next(nullptr)
6     {}
7 };

```

6.3.2 Adding Categories (Create)

```

1 void NameTable::add(int id, string name) {
2     int index = hashFunction(id); // Returns id % capacity
3     Node *current = table[index];
4
5     // Validation: Check for duplicates
6     while (current != nullptr) {
7         if (current->id == id) return; // Error: Duplicate ID
8         current = current->next;
9     }
10    // Head Insertion (O(1))
11    Node *newNode = new Node(id, name);
12    newNode->next = table[index];
13    table[index] = newNode;
14 }

```

How it works: The ‘add’ function uses a hash function (modulo operator) to map an integer ID to an index in the table array.

1. It traverses the linked list at that index to ensure the ID doesn’t already exist (collision handling).

2. If unique, it creates a new node and inserts it at the *head* of the linked list.

Why we chose a Hash Table: The Hash Table structure is ideal for the ID-Name mapping requirement.

- ****Direct Access:**** It allows for near-instant lookup of a Name given an ID, which is critical for the reporting features where thousands of transactions need to resolve their IDs to names.
- ****Sparse ID Support:**** Unlike a simple array where 'ID=1000' would require an array size of 1001, a hash table can store sparse IDs (e.g., 1, 50, 999) efficiently without wasting memory.
- ****Collision Handling:**** The separate chaining (linked list) approach ensures that even if two IDs hash to the same index, both can be stored and retrieved correctly.

6.3.3 Editing Categories (Update)

Allows renaming a category (e.g., "Food" → "Groceries") without breaking existing transactions.

```
1 void NameTable::editName(int id, string newName) {
2     int index = hashFunction(id);
3     Node* current = table[index];
4     while (current != nullptr) {
5         if (current->id == id) {
6             current->name = newName; // In-place update
7             return;
8         }
9         current = current->next;
10    }
11 }
```

How it works: Similar to 'add', it hashes the ID to find the correct bucket. It then traverses the list to find the node with the matching ID and updates its 'name' field directly. This validates our Transaction Management design: since transactions store IDs, they automatically link to this new name without any changes to the transaction records themselves.

6.3.4 Deleting Categories (Delete)

```
1 void NameTable::remove(int id) {
2     int index = hashFunction(id);
3     Node* current = table[index];
4     Node* prev = nullptr;
5
6     while (current != nullptr) {
7         if (current->id == id) {
8             if (prev == nullptr)
9                 table[index] = current->next; // Removing Head
10            else prev->next = current->next; // Removing Middle/Tail
11
12            delete current; // Free memory
13            return;
14        }
15        prev = current;
```

```

16     current = current->next;
17 }
18 }

```

6.4 Statistics & Reporting

The system provides granular reports using linear aggregation algorithms.

6.4.1 Time-based Statistics

Calculates total Income, Expense, and Net Balance within a user-defined date range.

```

1 TimeReport getStatTimeBased(date fromDate, date toDate, IncomeArray&
  incomes, ExpenseArray& expenses) {
2     TimeReport report = {0, 0, 0};
3
4     for (int i = 0; i < incomes.getCount(); i++) {
5         IncomeTransaction t = incomes.getAt(i);
6         if (compareDate(t.day, fromDate) >= 0 && compareDate(t.day,
  toDate) <= 0) {
7             report.totalIncome += t.amount;
8         }
9     }
10
11     repeat for expenses...
12
13     report.netBalance = report.totalIncome - report.totalExpense;
14     return report;
15 }

```

How it works: This function performs a linear scan over the entire transaction history. For each record, it checks if the date falls within the user-specified range. If it does, the amount is added to the running total.

Why this approach: While a database would use an index for this, for an in-memory application, a linear scan is extremely fast and simple to implement. It avoids the complexity of maintaining a sorted tree structure for dates.

Time complexity: $O(N + M)$, where N is the number of income records and M is the number of expense records. Since the data is stored in contiguous dynamic arrays, the algorithm benefits from high CPU cache locality, ensuring rapid execution even with a large dataset.

Net Balance Logic: The system computes the financial health metric directly within the loop:

$$\text{Net Balance} = \sum \text{Income} - \sum \text{Expense}$$

Encapsulation: Returning a single `TimeReport` object instead of using multiple reference parameters keeps the function signature clean and allows for easy data transfer to the UI layer.

```

1 struct TimeReport {
2     double totalIncome;

```



```

3     double totalExpense;
4     double netBalance;
5 };

```

6.4.2 Time & Wallet-based Statistics

Filters totals by specific Wallet ID within the current month (or range).

```

1 WalletStatRecord* IncWalletBased(date fromDate, date toDate, IncomeArray
  & incomes, WalletTable& wallets, int& count) {
2
3     StatRecord* tempRecords = new StatRecord[100];
4     int countInc = 0;
5
6     for (int i = 0; i < incomes.getCount(); i++) {
7         IncomeTransaction t = incomes.getAt(i);
8         if (compareDate(t.day, fromDate) >= 0 && compareDate(t.day,
  toDate) <= 0) {
9             updateStatRecord(tempRecords, countInc, t.walletID, t.amount
  );
10        }
11    }
12
13
14    count = countInc;
15    WalletStatRecord* result = new WalletStatRecord[count];
16
17    for (int i = 0; i < count; i++) {
18        result[i].walletName = wallets.getWalletName(tempRecords[i].id);
19        result[i].amount = tempRecords[i].amount;
20    }
21
22    delete[] tempRecords;
23    return result;
24 }

```

- **Primary Functions:** IncWalletBased (Income) and ExpWalletBased (Expense).
- **Helper Function:** updateStatRecord (core aggregation logic).
- **Return Type:** WalletStatRecord* (dynamic array of wallet statistics).

How it works: This function implements a "Group By" operation manually.

1. **Aggregation Phase:** A temporary array `tempRecords` is used as a lightweight *Map*. The algorithm iterates through filtered transactions and calls `updateStatRecord`. This helper function is responsible for:
 - Checking whether a wallet already exists in the array
 - Updating statistics if it exists
 - Adding a new record if it does not exist
2. **Mapping & Joining Phase:** Once aggregation is complete, the function dynamically allocates the result array based on the exact number of active wallets. It

then performs a **Data Join** using the `walletID` to retrieve the human-readable `WalletName` from the `WalletTable`.

Why this approach: To adhere to the "No STL Containers" rule (avoiding `std::map`), we implemented a custom aggregation logic using simple arrays.

Space Complexity: $O(K)$ where K is the number of active wallets. Since users rarely have more than 10–20 wallets, the temporary buffer is negligible in size.

Data Normalization: By aggregating based on `int walletID` rather than `string name`, we avoid expensive string comparisons ($O(L)$) inside the loop. Integer comparison is $O(1)$, significantly optimizing the aggregation process.

6.4.3 Annual Breakdown (Income/Expense)

Aggregates totals by Source/Category for multiple selected years to analyze spending habits.

```
1 BreakdownReport incomeAnnualBreakdown(int* selectedYears, int n,
2   IncomeArray& incomes, IncomeSourceTable& sources) {
3   BreakdownReport report;
4   report.totalAmount = 0;
5   report.count = 0;
6
7   StatRecord* tempRecords = new StatRecord[100];
8   int incCount = 0;
9
10  for (int i = 0; i < incomes.getCount(); i++) {
11    IncomeTransaction t = incomes.getAt(i);
12    if (isYearSelected(t.day.year, selectedYears, n)) {
13      report.totalAmount += t.amount;
14      updateStatRecord(tempRecords, incCount, t.sourceID, t.amount);
15    }
16  }
17
18  report.items = new BreakdownItem[incCount];
19  report.count = incCount;
20
21  for(int i = 0; i < incCount; i++) {
22    report.items[i].name = sources.getSourceName(tempRecords[i].id);
23    report.items[i].amount = tempRecords[i].amount;
24
25    if (report.totalAmount > 0)
26      report.items[i].percentage = (report.items[i].amount /
27      report.totalAmount) * 100.0;
28    else
29      report.items[i].percentage = 0;
30  }
31
32  delete[] tempRecords;
33  return report;
34 }
```

- **Primary Functions:** `incomeAnnualBreakdown` / `expenseAnnualBreakdown`.

- **Return Type:** `BreakdownReport` (contains total amount of earning/spending, the amount of income sources/expense categories and a dynamic array of `BreakdownItem`).

How this works:

Multi-Year Filtering: It uses a helper function `isYearSelected()` to check if a transaction belongs to the list of years requested by the user.

Source Aggregation: Similar to the wallet logic, it aggregates totals based on `categoryID` or `sourceID`.

Two-Pass Algorithm:

- **Pass 1:** Filter transactions and aggregate totals into temporary records to determine the number of unique categories or sources.
- **Pass 2:** Allocate the `BreakdownItem` array and populate it with data.

Distribution Calculation: It calculates the percentage share of each category:

$$Percentage = \left(\frac{CategoryTotal}{GrandTotal} \right) \times 100$$

Final Output: The function returns a fully populated `BreakdownReport` structure, which serves as the final output for frontend synchronization.

- **Total Amount:** The total income or expense aggregated across all selected years.
- **Item Count:** The number of unique categories or sources included in the breakdown.
- **Breakdown Items Array:** A dynamic array of `BreakdownItem`, where each item represents:
 - The income or expense amount for a specific category or source
 - The human-readable source or category name
 - The percentage contribution relative to the total income or expense

This structured output is designed to be directly consumed by the frontend layer, enabling seamless visualization (e.g., charts, tables) without requiring additional client-side computation.

Why this approach:

Analytical Application: This implements the Pareto Principle analysis logic, helping users identify which 20% of categories account for 80% of their spending.

Precision: The system uses `double` for calculation but formats output using `std::setprecision(1)` to provide user-friendly, readable reports (e.g., "15%") without losing underlying data accuracy.

Accuracy: The logic includes a check for `TotalAmount > 0` to prevent division-by-zero errors, ensuring program stability.

Efficiency: The two-pass design allocates memory exactly once and minimizes unnecessary computation on the frontend.

Frontend Sync: The `BreakdownReport` acts as a complete data contract, allowing the frontend to render visualizations directly without extra processing.

6.4.4 Detailed Transaction Log (History)

Function: `viewTransactionHistory`

How it works:

This function acts as a **Data Viewer**. It iterates through the arrays and prints a formatted table. Crucially, it performs **Real-time Data Joining**:

- It takes the `sourceID` (from `Transaction`) → looks up `IncomeSourceTable`.
- It takes the `walletID` (from `Transaction`) → looks up `WalletTable`.

Why this approach:

Decoupling Logic: This validates the system design where storage is decoupled from presentation. The transaction records remain compact (storing only `int` IDs), while the view layer handles the reconstruction of meaningful data.

Maintainability: If a user renames a Wallet in the master settings, this history view updates automatically without needing to modify any transaction records, ensuring **Data Consistency**.

6.4.5 Architecture: Data Synchronization Frontend Integration

A critical architectural decision in the `Statistic` module is the strict use of **Non-Void Return Types** for all computation functions.

How It Works: Instead of printing results directly to the console (using `std::cout`) inside the calculation functions, every function returns a specific **Data Transfer Object (DTO)** defined in the header file (e.g., `TimeReport`, `BreakdownReport`, `WalletStatRecord`).

Why This Approach:

- **Separation of Concerns:** This adheres to the software engineering principle of decoupling *Business Logic* (backend processing) from *Presentation Logic* (frontend UI). The calculation module does not need to know how the data will be displayed.
- **Frontend Synchronization:** The UI layer (managed by Member 2) receives a structured object containing processed data. This allows the frontend to freely decide how to render the data (e.g., as a formatted table, a list, or a dashboard) without modifying the core calculation code.
- **Testability:** Returning specific values makes unit testing significantly easier. We can programmatically verify the correctness of the returned data structures (checking struct fields) without the complex overhead of capturing and parsing console output strings.

6.5 Data Persistence

The system ensures no data loss via Binary File Storage and automated Session Persistence.

6.5.1 Binary Serialization (File Storage)

We utilize "std::fstream" in "std::ios::binary" mode. This is more efficient and secure than text files.

```
1 void IncomeArray::saveToFile(const string &filename) {
2     ofstream fout(filename, ios::binary);
3
4     // 1. Write Header (Count)
5     fout.write((char*)&count, sizeof(int));
6
7     for (int i = 0; i < count; i++) {
8         // 2. Write POD types (Date, Amount, IDs)
9         fout.write((char*)&list[i].day, sizeof(date));
10        fout.write((char*)&list[i].amount, sizeof(double));
11        fout.write((char*)&list[i].sourceID, sizeof(int));
12        fout.write((char*)&list[i].walletID, sizeof(int));
13
14        // 3. Serialize String (Length + Content)
15        int descLen = list[i].description.size();
16        fout.write((char*)&descLen, sizeof(int));
17        fout.write(list[i].description.c_str(), descLen);
18    }
19    fout.close();
20 }
```

How it works: The function writes the raw bytes of the data structures directly to the disk.

1. It writes the total count of items first (metadata).
2. It iterates through the array. For Plain Old Data (POD) types like 'int' and 'double', it simply dumps the memory content.
3. For strings, which are dynamic, it writes the *length* of the string as an integer first, followed by the actual character array.

Why this approach: Binary serialization is chosen for performance and compactness. It avoids the overhead of converting numbers to text (e.g., storing the integer '1234567890' takes 4 bytes in binary vs. 10 bytes in text). It also implicitly handles special characters in descriptions without needing complex escaping rules like CSV.

6.5.2 Session Persistence (Loading Logic)

The system automatically reloads state on startup. The loader includes ****Defensive Programming**** to prevent crashes if a file is corrupted (e.g., negative string length).

```
1 void IncomeArray::loadFromFile(const string &filename) {
2     ifstream fin(filename, ios::binary);
3     if (!fin.is_open()) return; // Handle missing file gracefully
4
5     fin.read((char*)&count, sizeof(int));
```

```

6     if (count > capacity) { /* Resize Logic */ }
7
8     for (int i = 0; i < count; i++) {
9         // ... Read POD types ...
10
11        // Safe String Reading
12        int descLen;
13        fin.read((char*)&descLen, sizeof(int));
14
15        if (descLen < 0 || descLen > 10000) { // Safety Check
16            list[i].description = "";
17        } else {
18            list[i].description.resize(descLen);
19            fin.read(&list[i].description[0], descLen);
20        }
21    }
22    fin.close();
23 }

```

7 Testing

7.1 Testing Strategy

The testing phase focused on ensuring data integrity, system stability, and user interface responsiveness. Due to the interactive nature of the Graphical User Interface (GUI), the primary testing methodology was **Manual Black-Box Testing**, supplemented by **White-Box Testing** for critical backend algorithms.

The testing strategy was divided into three specific phases:

1. **Unit Logic Verification:** Individual backend components (e.g., `IncomeArray`, `RecurringManager`) were tested in isolation to verify memory management (resizing logic) and date calculations.
2. **Integration Testing:** Verification of the communication between the UI layer (`TransactionWindow`, `App`) and the `DataManager`. This ensured that user inputs were correctly parsed, validated, and stored in the data structures.
3. **System & Persistence Testing:** End-to-end scenarios involving application restart cycles to ensure binary serialization (`.dat` files) correctly preserved the state of the application.

7.2 Test Results

A series of test cases were executed to validate core functionalities and edge cases. Particular attention was paid to data constraints and validation logic.

Test ID	Feature	Steps / Input	Expected Result	Status
TC-01	Expense Validation	1. Open “Add Expense”. 2. Select Wallet with 50k balance. 3. Input Amount: 100k. 4. Click Add.	System displays Error Code 9: “Insufficient funds”. Transaction is rejected.	PASS
TC-02	Data Integrity (Delete)	1. Create Wallet “Bank”. 2. Add transaction linked to “Bank”. 3. Go to Master Data. 4. Attempt to delete “Bank”.	Deletion is blocked to prevent orphaned transactions. Console/UI reports error.	PASS
TC-03	Recurring Execution	1. Add Recurring Income (start: Today). 2. Submit form. 3. Return to Dashboard immediately.	Transaction appears in history immediately without requiring app restart.	PASS
TC-04	Dashboard Refresh	1. Note Total Balance. 2. Add Income (+1,000,000). 3. Return to Dashboard.	Dashboard updates immediately to reflect new total. No visual lag.	PASS
TC-05	Persistence	1. Add various data. 2. Close Application. 3. Re-open Application.	All Wallets, Categories, and Transactions load correctly. Recurring checks run on startup.	PASS
TC-06	Input Validation	1. Input “abc” into Amount field. 2. Input invalid date “32/01/2024”.	System detects format errors and prevents submission (Error Code 1 and 2).	PASS

7.3 Performance Optimization

During the testing phase, a critical performance issue was identified in the **Dashboard** rendering logic. Originally, the application recalculated financial totals every frame (60 times per second), leading to unnecessary CPU usage.

This was resolved by implementing a `RefreshDashboard()` mechanism in `App.cpp`, which only recalculates data when a state change occurs (e.g., adding a transaction). This resulted in stable frame times and eliminated memory leaks associated with repeated heap allocation in the render loop.

7.4 Known Issues and Limitations

Despite the successful implementation of core features, the following limitations exist in the current version:

- **Unicode/Vietnamese Character Support:** The custom `TextInput` component currently casts keyboard input to `char` (ASCII). Consequently, entering Vietnamese characters (e.g., `đ`, `ê`, `ã`) in descriptions may result in garbage text or missing characters. Full UTF-8 support requires integrating Raylib's `CodepointToUTF8` functions.
- **Fixed Window Resolution:** The application layout is hardcoded for a 1280×800 resolution. Resizing the window does not dynamically adjust the UI element positions or font sizes.
- **Binary File Versioning:** The custom binary serialization logic does not currently support version headers. If the `struct` definitions (e.g., `IncomeTransaction`) are modified in future code updates, existing save files will become incompatible and unreadable.
- **Security:** Financial data is stored in unencrypted binary files. While not human-readable in a text editor, the data is not secure against intentional tampering or extraction tools.

8 Future Improvements

While the current version of the Personal Finance Manager provides a robust foundation for tracking incomes, expenses, and recurring transactions, several areas have been identified for future development. These enhancements aim to improve system stability, user experience, and data security.

8.1 Technical Architecture & Refactoring

- **Adoption of Modern C++ Standards (RAII):** The current codebase relies on manual memory management using raw pointers. Future versions should transition to Smart Pointers (`std::unique_ptr` and `std::shared_ptr`) to automatically prevent memory leaks and strictly adhere to the Resource Acquisition Is Initialization (RAII) idiom.
- **Standard Template Library (STL) Integration:** Replacing custom data structures (like `NameTable` and `IncomeArray`) with standard containers such as `std::unordered_map` and `std::vector` would optimize performance and improve code maintainability.

8.2 User Interface (UI) and Experience

- **Full Unicode Support:** Integrating Raylib's `CodepointToUTF8` functionality is necessary to support multi-byte characters, allowing users to enter descriptions and names in Vietnamese (e.g., `đ`, `ê`, `ã`) without corruption.
- **Data Visualization:** The reporting system should be enhanced to include graphical representations, such as Pie Charts for expense categorization and Line Graphs for net balance trends over time.

- **Responsive Layout:** Implementing a dynamic layout manager that calculates positions based on window dimensions would allow users to resize the application window or use fullscreen mode effectively.

8.3 Functional Enhancements

- **Budgeting System:** A new module allowing users to set spending limits for specific categories (e.g., "Food limit: 5,000,000 VND") would add significant value. The system could trigger alerts when transactions exceed these thresholds.
- **Advanced Search and Export:** Adding a search bar to filter transaction history by keyword or date range, along with a feature to export data to .CSV or .XLSX formats, would greatly improve data accessibility.

8.4 Security and Persistence

- **Database Integration:** Migrating the storage backend from custom binary files to **SQLite** would provide atomic transactions, preventing data corruption during unexpected crashes.
- **Encryption & Authentication:** To protect sensitive financial data, future updates should include a login screen and implement AES encryption for local data files.

9 Demo Videos

- **Full Feature Demo:** <https://www.youtube.com/watch?v=onqQoX757z0>

10 Work Division

The workload was distributed among team members as follows:

Team Member	Responsibilities
Phạm Quốc Bảo	• Backend Architecture: Core logic for Transactions and Recurring Manager (IncomeTransaction.cpp, ExpenseTransaction.cpp, recurring.cpp). • Data Management: Designed NameTable structure and Binary Persistence (Save/Load). • System Reliability: Developed Console Interface (CLI), Logic Validation, and Technical Documentation. • Testing Support: Assisted with manual testing scenarios and bug reporting.
Nguyễn Quốc Bảo	• User Interface (GUI): Implementation of the graphical interface using Raylib. • Component Design: Created reusable UI widgets (Buttons, Dropdowns, ScrollAreas, TextInputs, Dialogs). • Visual Layout: Designed the Dashboard and Window layouts.
Lê Nguyễn Hải Đăng	• Statistic Reporting: Core logic for Statistic & Reporting features and helpers function (Developed statistic.cpp, date.cpp) • Testing Support: Assisted with testing Statistic & Reporting functions by Statistic Menu on Console.

11 Collaboration Tools Used

- **Github:** <https://github.com/simble2k/PersonalFinanceManager.git>